



Sharif University of Technology

Department of Electrical Engineering

Course:

ASIC/FPGA

Report:

ASIC/FPGA Project - Group 8

Weekly Report I

Submitted by:

Mohammadhossein Sabzalian

402101906

Sina Hashemi

402102668

Erik Hartouni

401102778

Ali Behrad

401101321

Instructor:

Dr. Baharvand

Department of Electrical
Engineering

Term: Spring 2026

Project Management and Team Division

Following an initial group alignment meeting, the overall architectural requirements and milestone goals for the OTP Controller project were thoroughly discussed. To optimize parallel progress while coordinating around the team members' midterms, responsibilities were structured into two primary technical domains:

- **RTL Design and Hardware Architecture:** Handled by **Mohammadhossein Sabzalian** and **Sina Hashemi**. Our tasks include state-machine mapping, control logic formulation, block diagram specification, and datapath integration.
- **Verification and Testbench Infrastructure:** Handled by **Erik Hartouni** and **Ali Behrad**. Their tasks focus on modeling the test infrastructure, standardizing verification flows, and preparing functional simulation scaffolds.

Others:

- **GitHub Repository:**
The GitHub repository has been initiated with the corresponding folder structure.
- **Server Synchronization:**
The server has been synchronized with the GitHub repository so that changes in one apply to the other as well.

System Architecture and FSM Overview

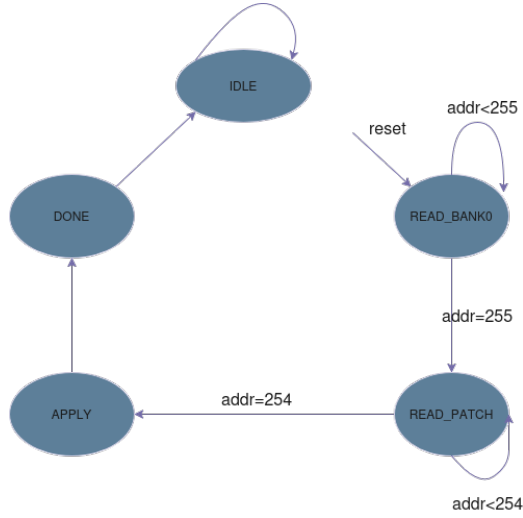


Figure 1: Finite State Machine (FSM) Transition Diagram

As illustrated in the FSM transition diagram (Figure 1), the system automatically enters the read sequence immediately following a hardware reset sequence:

1. **STATE_READ_BANK0**: The controller sequentially issues memory fetch operations from address 0 to 255 to read the raw baseline data. While evaluating the final bit, the transition criteria are satisfied, forcing the `next_state` to transition directly into the patch domain on the following clock edge.
2. **STATE_READ_PATCH**: During this state, the contents of the two potential patch records are serially shifted and stored within internal temporary registers.
3. **STATE_APPLY**: After performing a complete CRC integrity comparison, the system asserts its correction parameters, overriding the designated baseline fields using the validated patch records, and maps them to the output configuration registers.
4. **STATE_DONE** and **STATE_IDLE**: The `done` status flag is asserted for exactly one clock cycle. Afterward, the system stays anchored in the stable `IDLE` state. In this mode, both `done` and `busy` return to zero, freezing the output register states to prevent unnecessary dynamic power consumption across the broader design.

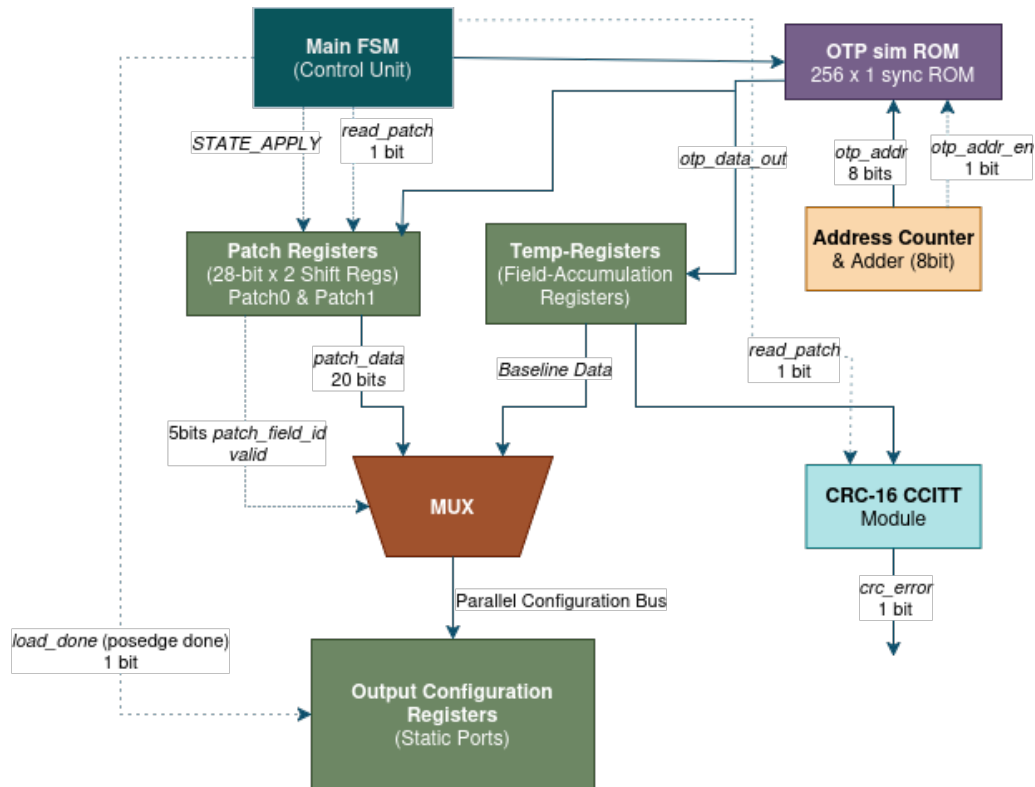


Figure 2: OTP Controller Core Block Diagram and Interconnects

Layout connections follow the diagrammed datapath in Figure 2, utilizing temporary field registers, a sequential address adder loop, an automated 16-bit CRC checksum block, and an override multiplexer fabric.

Implementation Progress and Code Snippets

The fundamental core components of the controller architecture have been implemented. The CRC checker utilizes an optimized sequential Galois-style parallel bitwise XOR tree to verify integrity over the first 183 loaded bits.

CRC-16 CCITT Module

Listing 1: CRC16 Hardware Implementation for Bitwise Stream Checksum

```
1 module crc16_ccitt (
2     input  wire [15:0] crc_in,
3     input  wire      data_in,
4     output wire [15:0] crc_out
5 );
6     wire xor_bit = data_in ^ crc_in[15];
7
8     assign crc_out[15] = crc_in[14] ^ xor_bit;
9     assign crc_out[14] = crc_in[13];
10    assign crc_out[13] = crc_in[12];
11    assign crc_out[12] = crc_in[11] ^ xor_bit;
12    assign crc_out[11] = crc_in[10];
13    assign crc_out[10] = crc_in[9];
14    assign crc_out[9]  = crc_in[8];
15    assign crc_out[8]  = crc_in[7];
16    assign crc_out[7]  = crc_in[6];
17    assign crc_out[6]  = crc_in[5];
18    assign crc_out[5]  = crc_in[4] ^ xor_bit;
19    assign crc_out[4]  = crc_in[3];
20    assign crc_out[3]  = crc_in[2];
21    assign crc_out[2]  = crc_in[1];
22    assign crc_out[1]  = crc_in[0];
23    assign crc_out[0]  = xor_bit;
24 endmodule
```

- **Galois-Style Feedback Network:** Implements a sequential bitwise hardware XOR mapping for the standard CRC-16 CCITT polynomial ($G(X) = X^{16} + X^{12} + X^5 + 1$).
- **Single-Cycle Parallel Shift:** Evaluates the incoming 1-bit stream against the current remainder in a single clock cycle, updating taps at indices 0, 5, 12, and 15 simultaneously while direct-routing the remaining bit slices.

Main Controller FSM Excerpt

Listing 2: Core FSM Sequential Register and Next-State Combinational Logic

```
1 // Sequential state transition block
2 always @(posedge clk or negedge rst_n) begin
3     if (!rst_n)
4         state <= STATE_READ_BANK0;
5     else
```

```

6         state <= next_state;
7     end
8
9     // Combinational next-state determination
10    always @(*) begin
11        case (state)
12            STATE_IDLE:      next_state = STATE_IDLE;
13            STATE_READ_BANK0: next_state = (addr == 8'd255) ?
14                STATE_READ_PATCH : STATE_READ_BANK0;
15            STATE_READ_PATCH: next_state = (addr == `PATCH1_HI) ?
16                STATE_APPLY : STATE_READ_PATCH;
17            STATE_APPLY:      next_state = STATE_DONE;
18            STATE_DONE:       next_state = STATE_IDLE;
19            default:          next_state = STATE_IDLE;
20        endcase
21    end

```

- **Automatic Reset Bootstrapping:** Bypasses manual entry strobes by directly initializing into STATE_READ_BANK0 upon an asynchronous active-low reset assertion to begin autonomous memory loading.
- **Deterministic Sequence Boundary Check:** Coordinates transition parameters strictly using relative hardware pointer positions, enforcing clean state handshakes based on address index markers (e.g., reaching 8'd255 or PATCH1_HI).

Data Accumulation and Deserialization Logic

Listing 3: Serial Stream Demultiplexing and Storage Vector Reconstruction

```

1 // Bit-by-bit serial data reconstruction from OTP stream
2 if (otp_read_en) begin
3     if (addr >= `OTP_RO_TRIM_LO && addr <= `OTP_RO_TRIM_HI)
4         tmp_ro_trim[addr - `OTP_RO_TRIM_LO] <= otp_data_out;
5     if (addr >= `OTP_CRC_LO && addr <= `OTP_CRC_HI)
6         tmp_crc_stored[addr - `OTP_CRC_LO] <= otp_data_out;
7 end

```

- **Dynamic Relative Indexing:** Dynamically maps a 1-bit serial input line (otp_data_out) into multi-bit parallel registers by subtracting field-specific lower bounds (LO) from the active address pointer.
- **Modular Resource Allocation:** Collects configurations and validation fields concurrently into temporary storage matrices, avoiding multi-driver conflicts prior to the final patch application process.

Patch Validation and Dynamic Override Logic

Listing 4: Conditional Redundancy Checking and Output Parameter Remapping

```

1 wire patch0_valid = patch0_valid_bit && (patch0_tag == `PATCH_TAG_VAL);

```

```
2
3 // Conditional hardware override execution inside STATE_APPLY
4 if (patch0_valid) begin
5     case (patch0_id)
6         `PID_RO_TRIM:    ro_trim_code <= patch0_data[14:0];
7         `PID_CFG_FLAGS:  cfg_flags    <= patch0_data[5:0];
8         default: ;
9     endcase
10 end
```

- **Hardware Integrity Guard:** Features dual-layer protection combining a single-bit activation register flag with an exact 2-bit permanent validation tag match (2'b10) to eliminate spurious correction triggers.
- **Hot-Swappable Configuration Remapping:** Implements an override multiplexer structure inside STATE_APPLY that cleanly hot-swaps faulty hardware blocks with the corresponding post-trim data vectors.

Challenges, Inter-Group Standardization and Future Plans

Engineering Ambiguities Resolved During hardware definition mapping, clear ambiguities arose regarding structural assignments within the configuration layout (e.g., bit arrangements of OE_MODE, specific CFG_FLAGS, and index bounds of individual patch fields).

To resolve these ambiguities, specific architectural assumptions were formally established in our design mapping:

Configuration Flag Bit Mapping:: Within the 6-bit CFG_FLAGS field (allocated at OTP addresses 77 to 82), bit 0 is strictly mapped to `oe_mode`, bit 1 to `oe_pol`, and bit 5 to the `prog_disable_lock` flag[cite: 119, 123].

Memory Collision Resolution: To eliminate structural overlap between the baseline configuration data and the redundancy payload, the patch storage window was explicitly positioned to begin right after the execution boundary of the CRC-16 checksum module[cite: 121, 122]. Specifically, Patch 0 is mapped immediately following the CRC segment from address 196 to 223, ensuring safe sequential access without data corruption[cite: 121, 122].

To guarantee flexibility and structural compliance, we synchronized coordinates directly with the project TA and established an explicit convention framework with **Group 7**. This standardizes bit mappings, guaranteeing that our verification environment can successfully interact with external multi-group components.